Custom Reinforcement Learning Agent Implementation using Farama Gymnasium

Student ID: Fa23-bai-039 | Environment: CliffWalking-v1 | Algorithm: Tabular Q-Learning (from scratch)

1. Problem Description

This work implements and evaluates a tabular Q-Learning agent on the Farama Gymnasium **CliffWalking-v1** benchmark. The environment is a 4×12 grid world (48 discrete states). The agent starts at the bottom-left cell and must reach the goal at the bottom-right cell. Between start and goal lies a cliff: stepping into any cliff cell incurs a large penalty and resets the agent to the start. The action space is discrete with four moves: **0=Up, 1=Right, 2=Down, 3=Left**.

Reward structure: each transition yields −**1** (step cost). Falling off the cliff yields −**100** and returns the agent to the start (episode continues until the goal or truncation). Reaching the goal terminates the episode. An optimal safe path along the cliff edge has return −13 (13 steps). A safer but longer detour yields a more negative return. This creates a classic exploration–safety trade-off that makes CliffWalking a strong testbed for on-policy vs off-policy learning behavior.

2. Methodology

2.1 Algorithm Formulation

We use **ε-greedy Q-Learning**, an off-policy temporal-difference control algorithm. A Q-table $Q(s,a) \in \blacksquare^{|S|\times|A|}$ estimates action values. At each step the agent selects a random action with probability ε (exploration) and otherwise selects $arg\ max_a\ Q(s,a)$ (exploitation). After observing transition (s,a,r,s'), the update is:

$$\mathbf{Q(s,a) \leftarrow Q(s,a) + \alpha\ [\ r + \gamma\ \max_{a'} Q(s',a') - Q(s,a)\ ]}$$ (terminal: bootstrap target = r)

Here α is the learning rate, γ the discount factor, and ε decays exponentially each episode: $\varepsilon \leftarrow max(\varepsilon_{min}, \varepsilon \cdot \varepsilon_{decay})$. Because the bootstrap uses $max_{a'} Q(s',a')$ rather than the behavior action, Q-Learning can learn the optimal policy while still exploring.

2.2 Implementation Architecture

The codebase is modular: **agent.py** contains *QLearningAgent* (action selection, TD update, ε-decay, save/load); **utils.py** provides environment creation, the training loop, evaluation, plotting, GIF rendering, and a hyperparameter sweep; **main.py** is the CLI entry point that wires training, evaluation, artifact export, and sensitivity analysis. The environment is created with *gymnasium.make("CliffWalking-v1")*. Trained weights are stored as *models/q_table.npy* (and *.npz* with hyperparameters). A short rgb_array rollout is saved as *results/agent_demo.gif*.

3. Experimental Setup & Hyperparameters

All experiments use Python 3 with Gymnasium. The final tuned configuration (used for the saved model and demo GIF) is listed below. Sensitivity runs train for 3,000 episodes under identical seeds while varying one factor at a time.

Hyperparameter	Symbol	Final Value	Notes
Learning rate	α	0.5	Faster than 0.1; stable vs 0.9
Discount factor	γ	0.99	Prefers near-optimal short paths
Initial epsilon	ε■	1.0	Fully exploratory start
Minimum epsilon	ε_min	0.01	Residual exploration
Epsilon decay	ε_decay	0.995	Per-episode multiplicative
Training episodes	N	5000	Main training run
Max steps / episode	T	200	Safety truncation
Evaluation episodes	—	100	Greedy policy
Random seed	—	42	Reproducibility

4. Results & Discussion

After 5,000 training episodes, greedy evaluation over 100 episodes achieved a mean return of −**13.0 ± 0.0** with a **100%** success rate — matching the optimal safe path length. Training curves show rapid improvement within the first

~1–2k episodes, then stabilization as ε decays. The learned value heatmap concentrates higher values along the path to the goal and lower values near cliff cells, indicating the agent internalized the cliff penalty.

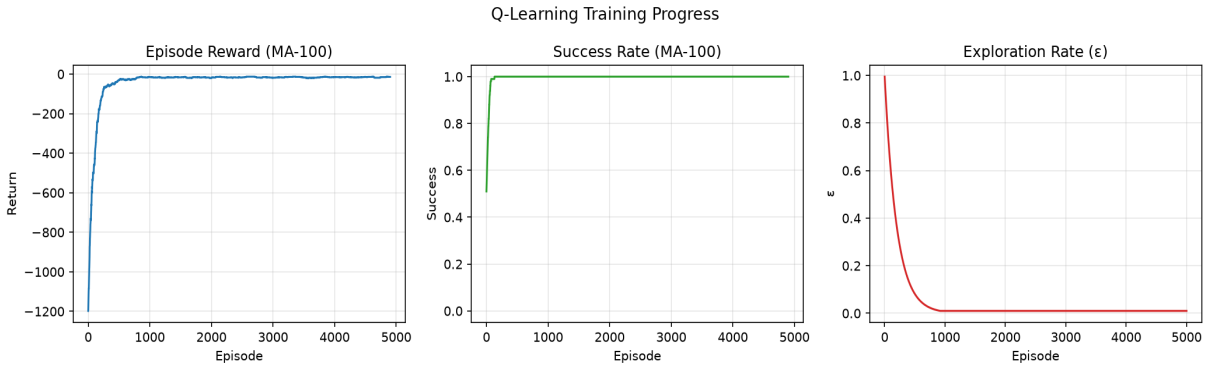


Figure 1. Training reward, success rate, and ε decay.

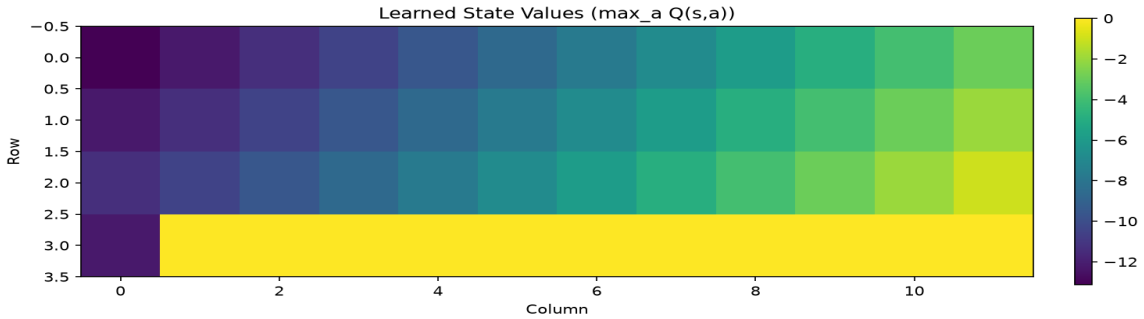


Figure 2. Learned state values (max_a Q(s,a)) on the 4×12 grid.

Hyperparameter sensitivity. Moderate learning rates ($\alpha \approx 0.5$) outperform very small α (slow learning within a fixed episode budget) and can be more stable than very large α (noisy updates). High γ (0.99) encourages valuing future goal attainment over myopic step-cost minimization alone. Slower ε decay prolongs exploration: helpful early, but it can delay convergence of the reported greedy evaluation if too much random walking persists late in training.

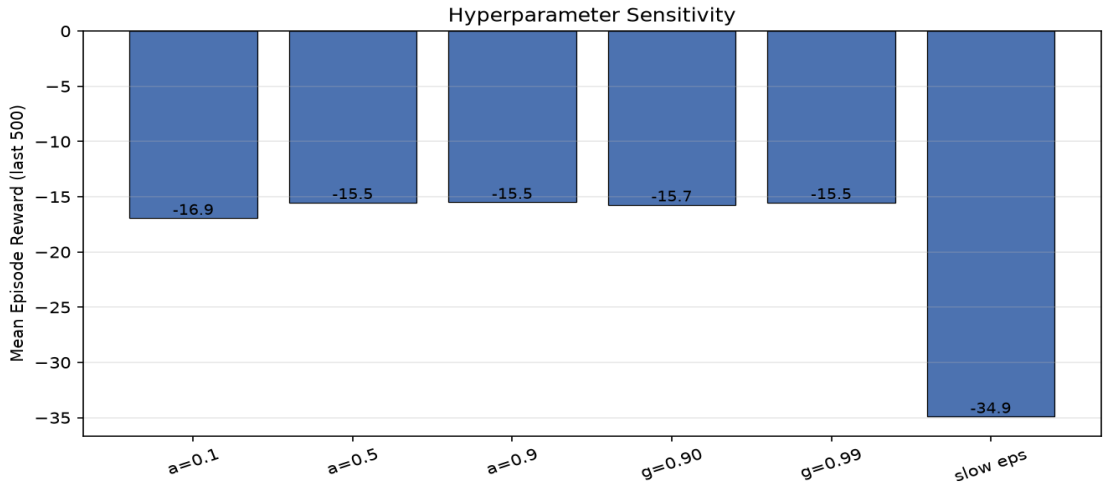


Figure 3. Mean return over the last 500 episodes for each hyperparameter configuration.

4.1 Failure Modes & Observations

(1) **Cliff falls during exploration:** high ε causes frequent -100 penalties early on; this is expected and necessary for discovering cliff boundaries. (2) **Suboptimal long detours:** if exploration is insufficient near the cliff edge, the agent may converge to a safer upper-row path with worse return than the optimal -13 path. (3) **Maximization bias:** standard Q-Learning can overestimate action values; Double Q-Learning would be a natural extension. (4) **Discrete tabular limit:** this solution does not transfer to continuous control (e.g., CartPole) without function approximation (DQN/Policy Gradient).

Qualitatively, the rendered GIF (*results/agent_demo.gif*) shows the greedy policy navigating from start to goal without falling, confirming that the learned policy solves the task under evaluation conditions.

5. Conclusion

We implemented a complete from-scratch Q-Learning pipeline on Gymnasium CliffWalking-v1, including modular training code, reproducible hyperparameters, evaluation metrics, training visualizations, a saved Q-table, and a policy GIF. The agent converges to a successful goal-reaching policy. Hyperparameter analysis confirms that α, γ, and ε-decay materially affect sample efficiency and final return. Future work includes Double Q-Learning, SARSA (safer on-policy cliff behavior), and deep RL on continuous Gymnasium tasks.

Artifacts: source (*main.py*, *agent.py*, *utils.py*), notebook (*notebooks/training.ipynb*), model (*models/q_table.npy*), figures/GIF (*results/*).